SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*
Assessment Tool Weightage: 50%

CO2_AT1_Scenario-Based Assessment

Code of Conduct:

I Soorys S / 192311060 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
SooryaS

Answers:

1. An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification. (5 Marks)

Answer:

Frequent failures near transfer limits indicate inadequate boundary testing. The team may have tested ordinary amounts without systematically checking the minimum and maximum permitted values.

1. Equivalence Partitioning (ECP): If the allowed transfer is ₹100–₹50,000, divide inputs into below minimum (invalid), ₹100–₹50,000 (valid), and above maximum (invalid).
2. Boundary Value Analysis (BVA): Test values immediately below, at, and above both limits.

Test cases (assuming these illustrative limits):
- ₹99: Reject as below minimum.
- ₹100: Accept if all other conditions pass.
- ₹101: Accept if all other conditions pass.
- ₹49,999 and ₹50,000: Accept if other conditions pass.
- ₹50,001: Reject as above maximum.
- Zero and negative amounts: Reject.

Also test insufficient balance, daily transfer limits, decimal precision, duplicate submissions, and transaction rollback. Random testing can discover unexpected cases but may miss exact boundaries and be difficult to reproduce. Retain it as supplementary fuzz/random testing, not as the primary approach. Automate ECP, BVA, decision-table, and regression tests; record failures and verify account balances and audit logs. This structured approach improves defect detection while avoiding redundant tests.

2. A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk. (5 Marks)

Answer:

Critical defects after deployment indicate that important decision outcomes, exception paths, or input conditions were not tested. Statement coverage alone is insufficient because every statement may execute without testing both outcomes of each decision.

Conceptual Control Flow Graph (CFG):
Start → Validate input
• Valid → Check clinical rule → Rule met: save result → End
• Valid → Check clinical rule → Rule not met: return warning → End
• Invalid → return validation error → End

Coverage analysis:
- Statement coverage checks whether each statement executes.
- Branch coverage checks true and false outcomes of each decision.
- Path coverage checks execution paths but exhaustive path coverage may be impossible when loops or many conditions exist.

Improved strategy:
1. Draw a CFG including normal, error, and exception flows.
2. Design tests for every branch and independent basis path.
3. Include missing patient data, invalid values, database failures, timeouts, and boundary cases.
4. Use coverage tools to identify untested branches.
5. Run regression tests and validate expected clinical outcomes with domain experts.

This strategy finds logic and failure-handling defects earlier, improves reliability, and reduces risk to patients. Coverage supports confidence but does not by itself prove correctness.

3. A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution. (5 Marks)

Answer:
Testing each field separately can miss interaction errors, such as discounts exceeding the price or zero quantity combined with otherwise valid inputs. Exhaustively testing every combination may be costly, so several black-box techniques should be combined.

Assume price must be greater than ₹0, quantity must be an integer from 1 to 100, and discount must be 0–50%. These limits are illustrative and should be replaced by actual requirements.

ECP and BVA:
- Price: test invalid values ≤ 0 and valid values > 0; include −₹1, ₹0, and the smallest valid positive amount.
- Quantity: valid 1–100; test 0, 1, 2, 99, 100, and 101.
- Discount: valid 0–50%; test −1%, 0%, 1%, 49%, 50%, and 51%.

Decision-table cases:
- All inputs valid → calculate total correctly.
- Invalid price → reject price.
- Invalid quantity → reject quantity.
- Invalid discount → reject discount.
- Multiple invalid inputs → reject and report appropriate validation errors.

Also test maximum quantity with maximum discount, currency rounding, tax calculations, and discount caps. Pairwise testing can cover pairs of parameter values with fewer tests, while high-risk combinations should be tested explicitly. ECP reduces redundant tests, BVA targets edge defects, and decision tables detect interaction errors.

4. A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner. (5 Marks)

Answer:
Partial path coverage suggests that some branches, loop exits, exception paths, or condition combinations have not been tested. An incomplete CFG may omit loop back-edges, break/continue statements, or error-handling routes.

Testing strategy:
1. Construct a CFG containing every decision, loop entry/exit, exception handler, and return.
2. Calculate cyclomatic complexity: $V(G) = E - N + 2P$, where E is the number of edges, N is the number of nodes, and P is the number of connected components. For one connected graph, $V(G) = E - N + 2$.
3. Identify a basis set of independent paths. For one connected graph, the number of basis paths equals V(G).
4. Create test data to execute each basis path and both outcomes of every decision.
5. Test loops for zero iterations where valid, one iteration, typical iterations, and boundary or maximum iterations.
6. Measure branch and basis-path coverage and add risk-based tests for important condition combinations.

Exhaustive path testing is usually impractical with loops because the number of paths may be very large or infinite. Basis-path testing combined with loop-boundary and exception testing is more feasible. It helps detect incorrect loop conditions, missing exits, and errors in nested logic.

5. A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box

methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution. (5 Marks)

Answer:

Production defects despite using both methods may indicate poor requirements traceability, missing boundary tests, untested exception paths, or over-reliance on a single coverage metric.

Hybrid strategy:

1. Use ECP and BVA for transaction amounts, fees, interest, account limits, and permissions.
2. Create decision tables for combinations such as sufficient balance, valid beneficiary, daily limit, and authentication status.
3. Construct CFGs for authentication, balance checks, posting, rollback, and exception handling.
4. Use branch and basis-path testing to exercise internal decision outcomes.
5. Test duplicate requests, concurrent transfers, network failures, partial database failures, and rollback.
6. Measure requirement coverage, boundary coverage, branch coverage, and risk-based path coverage.
7. Automate regression tests and reconcile account balances and audit logs.

Black-box testing checks externally visible requirements without depending on code structure. White-box testing reveals internal branches and paths that may otherwise remain unexecuted. Combining them provides complementary coverage. Security testing, code review, realistic test data, and production monitoring further reduce residual risk. No single metric guarantees defect-free software.

6. A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic. (5 Marks)

Answer:

Errors at exactly 5, 10, and 15 years suggest off-by-one errors, incorrect comparison operators, or ambiguity in how completed service years are calculated. Exploratory testing may discover such defects, but does not guarantee repeatable coverage of each threshold.

Assuming bonus tiers change at 5, 10, and 15 completed years, apply BVA:

- Around 5 years: test 4, 5, and 6.
- Around 10 years: test 9, 10, and 11.
- Around 15 years: test 14, 15, and 16.
- Test zero and negative service durations as invalid inputs.

ECP classes may be less than 5 years, 5 to less than 10 years, 10 to less than 15 years, and 15 years or more, depending on the documented rules. Test representative values in each class and combine them with every performance-rating category.

Also test service duration calculated from joining date, leap years, month-end dates, missing ratings, rounding, and total bonus calculation. ECP covers each rule class systematically, while BVA targets the exact thresholds where the failures occurred. Both techniques are repeatable and measurable, making them especially suitable for financial logic.

7. An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability. (5 Marks)

Answer:

The missed alarm suggests that a sensor-fault branch or a Boolean condition combination was not tested. Consider the decision D = A OR (B AND C), where A = sensor fault, B = temperature high, and C = cooling failed. If D is true, the alarm is triggered.

Conceptual CFG:

Start/read sensors → Check sensor fault
• Fault detected → Raise alarm and enter safe state → End
• No fault → Check high temperature
– Not high → Normal safe control → End
– High → Check cooling failure
* Failure → Raise alarm and enter safe state → End
* No failure → Continue controlled operation → End

A conceptual MC/DC test set is:

- A=F, B=F, C=F → D=F
- A=T, B=F, C=F → D=T (shows independent effect of A)
- A=F, B=T, C=F → D=F
- A=F, B=T, C=T → D=T (shows independent effect of C)
- A=F, B=F, C=T → D=F (paired with the previous case to show B's effect)

MC/DC requires each atomic condition to independently affect the overall decision outcome. The exact test set must match the implemented logic. Testing should include sensor disconnection, stale readings, threshold transitions, cooling failure, alarm timing, and safe-state behaviour. Instrument the code to measure condition and branch coverage. MC/DC is stronger than statement coverage and is valuable for safety-critical logic because it tests the independent influence of each condition.

8. A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative 'Total Amount' due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases. (5 Marks)

Answer:

Testing price, promo code, and user tier independently can miss interaction defects, such as stacking discounts beyond the price or applying a Premium-only code to a Free user.

Decision table (based on the stated business rules):

- Valid price + eligible promo + stacking permitted → apply permitted discounts; total must be ≥ 0.
- Valid price + eligible promo + stacking not permitted → reject the second code or apply only the allowed code.
- Valid price + ineligible promo for user tier → reject promo.
- Invalid price → reject booking input.
- Discounts exceed price → cap or reject according to documented policy; never produce a negative total.

Use ECP and BVA for price and discount limits, decision tables for promo/tier/stacking rules, and pairwise testing for remaining combinations. Explicitly test expired codes, maximum discounts, multiple codes, currency rounding, and API requests that bypass the UI. Verify totals against an independently calculated expected result.

Decision tables reveal interactions that isolated tests miss. Pairwise and risk-based selection reduce redundancy, while an invariant check ensures the final amount never violates the documented business rule.

9. A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent 'infinite loop' crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics. (5 Marks)

Answer:

Statement coverage only proves that statements execute at least once. It does not verify that loops terminate, exit conditions work, or empty and boundary-sized inputs are handled correctly.

Strategy:

1. Draw a CFG showing nested loops, loop back-edges, break/continue statements, retries, and exception exits.
2. Calculate cyclomatic complexity $V(G) = E - N + 2P$; for one connected CFG, $V(G) = E - N + 2$.
3. Identify a basis set of independent paths; the basis-path count equals $V(G)$ for one connected graph.
4. Test loops with zero iterations where permitted, one iteration, multiple iterations, and maximum supported records.
5. Test empty tables, one record, duplicate records, malformed data, database timeouts, and exceptions inside inner loops.

6. Add test-environment watchdog timeouts and iteration counters to detect non-termination safely.

Cyclomatic complexity helps estimate the number of independent paths needed for basis-path testing. It does not cover every possible loop repetition, so loop-boundary and stress tests remain necessary. This approach can reveal incorrect termination conditions and reduce infinite-loop failures during migration.

10. A mobile food delivery app features a 'Real-time Tracker' that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows. (5 Marks)

Answer:

ECP and BVA test input classes and limits but may miss invalid event sequences and missing transitions. State Transition Testing checks how the application changes state when events occur.

Example transitions:

- Ordered + restaurant accepts → Preparing.
- Preparing + driver picks up → Out for Delivery.
- Out for Delivery + successful delivery → Delivered.
- Out for Delivery + driver cancels → Reassign driver or enter a defined cancellation/refund workflow.
- Preparing + restaurant cancels → Cancelled and notify the customer.
- Delivered + late cancellation event → Remain Delivered and reject the invalid transition.

Test each valid transition, invalid transitions, repeated cancellation events, duplicate notifications, network delays, and app restart/recovery. Define expected customer notifications, reassignment, refunds, and order history updates for each path.

State-transition testing directly targets sequence-dependent defects and prevents the tracker from becoming stuck in an undefined state. ECP and BVA remain useful for identifiers, timestamps, and payment amounts, but do not replace transition coverage.

11. A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, 'Brute Force' vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module. (5 Marks)

Answer:

Brute-force vulnerabilities indicate insufficient testing of repeated failures, rate limits, lockout paths, OTP replay, or alternate authentication routes. Success-path testing alone is not enough for security.

Hybrid strategy:
- Error Guessing (black box): try repeated wrong passwords, expired or replayed OTPs, rapid requests, parallel attempts, and changes in IP or device.
- ECP/BVA: test OTP just before, at, and after expiry; test attempts below, at, and above the configured rate limit.
- CFG/path analysis: map password checks, OTP verification, IP-risk checks, rate limiting, lockout, account recovery, and audit logging. Exercise true and false outcomes and exception paths.
- Verify that limits apply consistently across endpoints and sessions and cannot be bypassed by parallel requests.
- Check generic error messages, single-use OTP enforcement, secure logging, and alert generation.

Use controlled security tests and penetration testing. Monitor failure counters and ensure responses do not reveal whether a username exists. Error guessing finds realistic attack patterns, while path analysis identifies untested internal routes. Together they improve resistance to brute-force attacks.

12. An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions. (5 Marks)

Answer:
The failure suggests unclear expiry rules, timezone issues, or incorrect date comparisons. Tests may have covered ordinary dates but missed the exact transition boundary.

ECP date classes:
- Before expiration: test upgrade one day before expiry.
- At expiration boundary: test the exact expiry date/time and apply the documented inclusive/exclusive rule.
- After expiration: test one day after expiry and follow the renewal/reactivation policy.
- Invalid date range: expiry earlier than subscription start should be rejected or flagged.

Test each class across Free, Basic, and Premium tiers and representative course types. Add leap-day, month-end, timezone, delayed payment, failed payment, and duplicate-upgrade tests. A decision table should define access based on tier, course restriction, payment status, and subscription status.

Requirements must specify the authoritative timezone and whether access expires at the beginning or end of the stated date. ECP ensures representative coverage of date and subscription classes, while boundary tests target the exact transition. Decision-table and regression testing improve consistency and prevent access errors.

13. A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures. (5 Marks)

Answer:

Statement coverage may execute each line without testing every switch-case outcome or both outcomes of each Boolean decision. Sensor thresholds, invalid readings, and hardware output paths may therefore remain untested.

Improved strategy:

1. Build a CFG covering each switch case, default case, decision, sensor-error handler, and hardware output.
2. Test every switch value, including unsupported values and the default branch.
3. Apply branch coverage to execute true and false outcomes of every decision.
4. Apply condition coverage so each atomic Boolean condition becomes true and false at least once.
5. Test readings below, at, and above dimming thresholds, plus disconnected, stale, and out-of-range sensor data.
6. Use mocks for unit tests and hardware-in-the-loop tests for integration.

For example, for (ambient_light < threshold) AND (sensor_valid), test low light with a valid sensor, high light with a valid sensor, and low light with an invalid sensor. These tests check whether each condition affects the output as intended. Branch and condition coverage, combined with fault injection and hardware-integration tests, reduce unexpected behaviour.

14. A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing 'negative scenarios' for robust system performance. (5 Marks)

Answer:

The failure indicates that tests focused on ordinary positive weights and missed invalid boundaries. Zero or negative weight may cause incorrect shipping fees, invalid invoices, or downstream failures.

Assume valid weight must be greater than 0 and no more than a documented maximum Wmax. Robust BVA should include:

- −0.01 kg → reject as negative.
- 0 kg → reject as zero.
- Smallest valid positive weight → accept and calculate correctly.
- Values around any minimum charge threshold → verify correct price tier.
- Wmax → accept if permitted.
- Just above Wmax → reject or route to special handling.
- Missing or non-numeric weight → return a clear validation error.

Repeat relevant cases across destination zones and delivery speeds. Check units, decimal precision, rounding, and grams-to-kilograms conversion. Validate on both the client and server because APIs or corrupted records may bypass the interface.

Negative testing is essential because invalid values can be submitted accidentally or maliciously. Robust BVA systematically tests just-inside and just-outside limits, improving validation and calculation reliability.

15. A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability. (5 Marks)

Answer:

Random testing may not consistently generate prime-sized datasets or exercise the specific loop and parallel partitioning conditions causing the defect. Prime-sized inputs can expose remainder-handling errors, uneven work distribution, off-by-one indexing, or race conditions.

White-box strategy:

1. Construct a CFG for data partitioning, loop bounds, remainder handling, worker creation, synchronization, and error paths.
2. Calculate cyclomatic complexity $V(G) = E - N + 2P$; for one connected CFG, $V(G) = E - N + 2$.
3. Identify a basis set of independent paths.
4. Test sizes 0, 1, small primes such as 7 and 11, nearby composite sizes such as 8 and 12, and sizes around thread/chunk boundaries.
5. Compare parallel output with a trusted serial implementation or validated reference result.
6. Repeat with different thread counts and schedules to expose race conditions. Use assertions, race detectors, and memory checks.

Basis Path Testing exercises independent control-flow paths, but cannot alone prove mathematical correctness or eliminate concurrency defects. Reference comparisons, property-based tests, and stress tests are also needed. This combined approach targets the prime-size defect and improves numerical consistency and system stability.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification